CH4 initial

Zhang Zhenglin

Contents

Necessary libraries **1**

Load data **2**

Make sure all columns are in the correct variable class **3**

Convert all observations to common currency **3**

Linear interpolation function **3**

Linear interpolation **4**

Interpolation visualisation **4**

check to make sure code is functioning well 4

Correlation function (look at corr of Emission_g_CH4_ha_day with cumulative) **8**

Try function **9**

Try R2 and regression **10**

Function 10

Run dataset 10

What if we just drew a triangle from day 62? **12**

Join data 13

What is we used the linear regrssion of day 62 emission against seasonal cumulative to get a cumulative value? Actually it is the same answer as immediately above. 15

Necessary libraries

```r
library(knitr)
library(ggplot2)
theme_set(theme_bw())
library(emmeans)
library(multcomp)
library(PLS205)
library(lme4)
library(lmerTest)
library(multcompView)
library(car)
library(Rmisc)
library(dplyr) #https://r4ds.had.co.nz/ (Chapter 3, Chapter 5, look at filter and select)
# https://bookdown.org/ansellbr/WEHI_tidyR_course_book/
library(stringr)
library(data.table)
library(GGally)
library(formatR)
library(readxl)
library(FluxCalR)
library(tidyverse)
library(fuzzyjoin)
library(purrr)
library(data.table)
library(broom)
library(lubridate)
library(readxl)
library(openxlsx)
library(ggrepel)
library(gridExtra)
library(ggpubr)
library(cluster)
```

Load data

```r
master <- read_excel("../Data/Initial_Database.xlsx", sheet = 1)
str(master)
```

```
## tibble [250 x 10] (S3: tbl_df/tbl/data.frame)
##  $ Number                          : num [1:250] 1 1 1 1 1 1 1 1 1 1 ...
##  $ Emission_g_CH4_ha_day           : num [1:250] 0 0 0 2549 4369 ...
##  $ Days_After_Flooding             : num [1:250] 0.688 4.812 11.688 18.562 26.125 ...
##  $ Variety                         : chr [1:250] "M206" "M206" "M206" "M206" ...
##  $ Year                            : num [1:250] 2021 2021 2021 2021 2021 ...
##  $ Location                        : chr [1:250] "RES" "RES" "RES" "RES" ...
##  $ Cumulative_seaonal_kg_CH4_ha    : num [1:250] 477 477 477 477 477 477 477 477 477 477 ...
##  $ Original_Emission_g_CH4-C_ha_day: num [1:250] NA NA NA NA NA NA NA NA NA NA ...
##  $ Original_Cumulative_CH_4-C_kg_ha: num [1:250] NA NA NA NA NA NA NA NA NA NA ...
##  $ Notes                           : chr [1:250] NA NA NA NA ...
```

Make sure all columns are in the correct variable class

```
master$Number <- as.factor(master$Number)
```

Convert all observations to common currency

Linear interpolation function

Number column here is our biological replicate. One set of time resolved flux data for one cumulative flux value.

```
library(dplyr)

interpolate_emissions <- function(data,
                                  group_cols = c("Number", "Year", "Location", "Variety"),
                                  day_col = "Days_After_Flooding",
                                  emission_col = "Emission_g_CH4_ha_day") {

  data %>%
    arrange(across(all_of(c(group_cols, day_col)))) %>%
    group_by(across(all_of(group_cols))) %>%
    group_modify(~ {

      df_group <- .x

      # Remove NA emission rows for interpolation check
      valid_rows <- df_group %>%
        filter(!is.na(.data[[emission_col]]))

      # If fewer than 2 valid points → cannot interpolate
      if (nrow(valid_rows) < 2) {
        return(df_group)
      }

      # Full daily sequence
      full_days <- seq(min(valid_rows[[day_col]]),
                       max(valid_rows[[day_col]]),
                       by = 1)

      # Linear interpolation
      interp_vals <- approx(
        x = valid_rows[[day_col]],
        y = valid_rows[[emission_col]],
        xout = full_days,
        method = "linear"
      )

      # Create template from first row
      template <- df_group[1, ]
      template <- template[rep(1, length(full_days)), ]
```

```
      template[[day_col]] <- interp_vals$x
      template[[emission_col]] <- interp_vals$y

      template
    }) %>%
    ungroup()
}
```

Linear interpolation

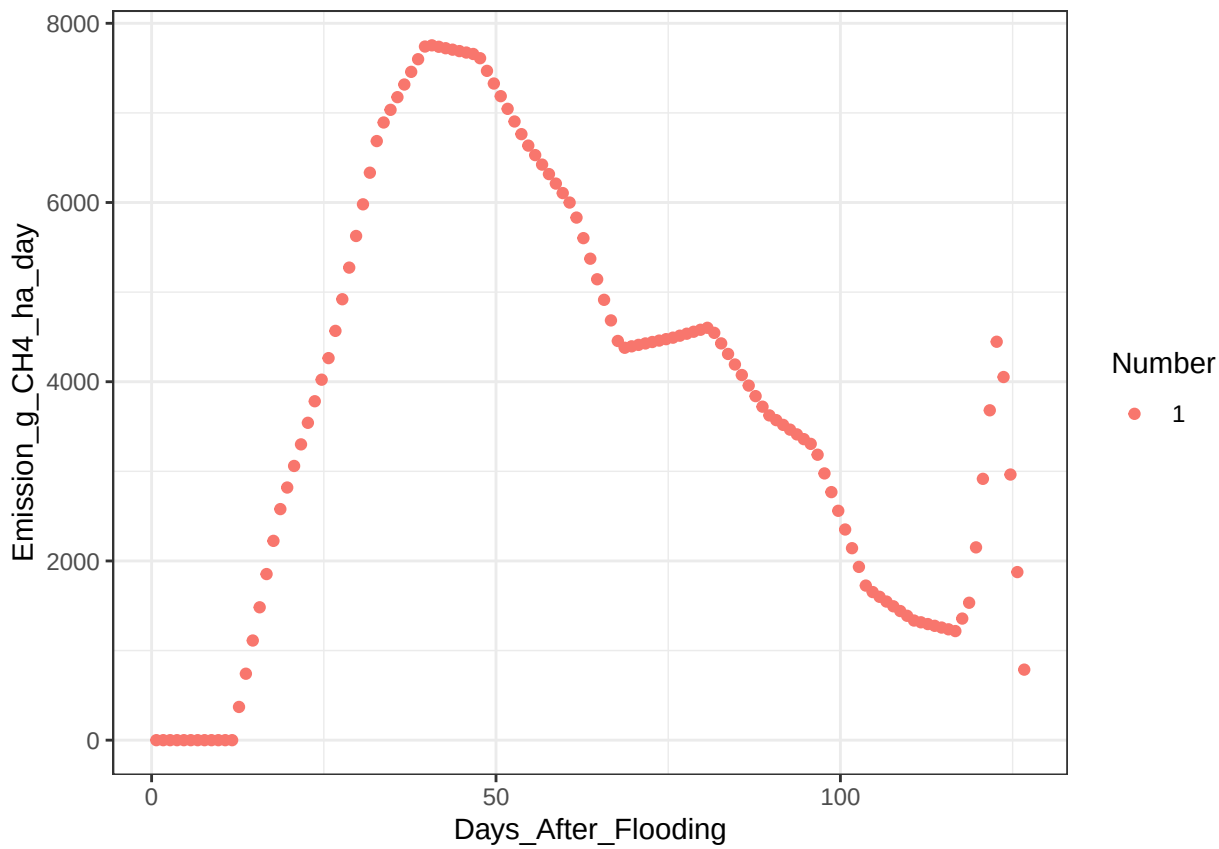
```
flux_interpolated <- interpolate_emissions(master) %>% mutate(Day_Int = as.integer(round(Days_After_Flo
```

Interpolation visualisation

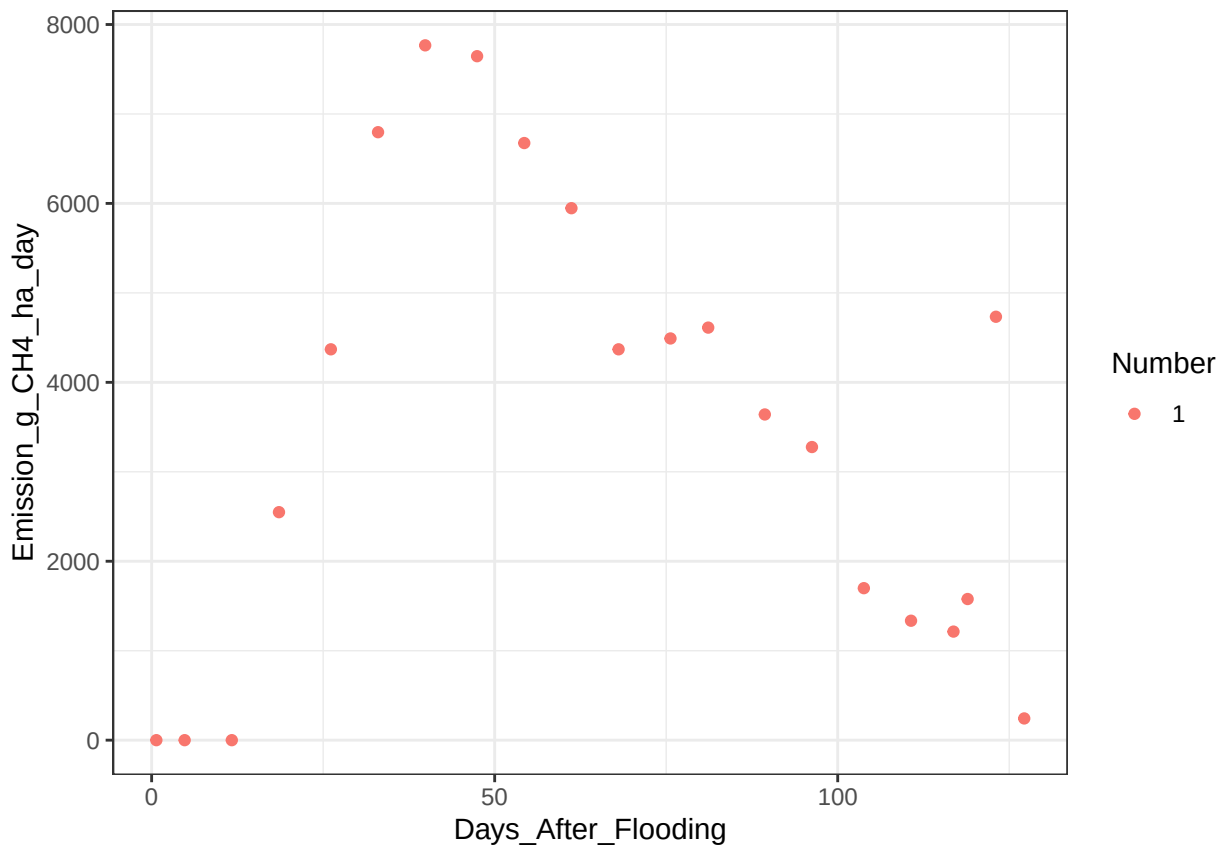
check to make sure code is functioning well

```
ggplot(flux_interpolated%>%filter(Number=="1"), aes(y=Emission_g_CH4_ha_day, x=Days_After_Flooding, col
```

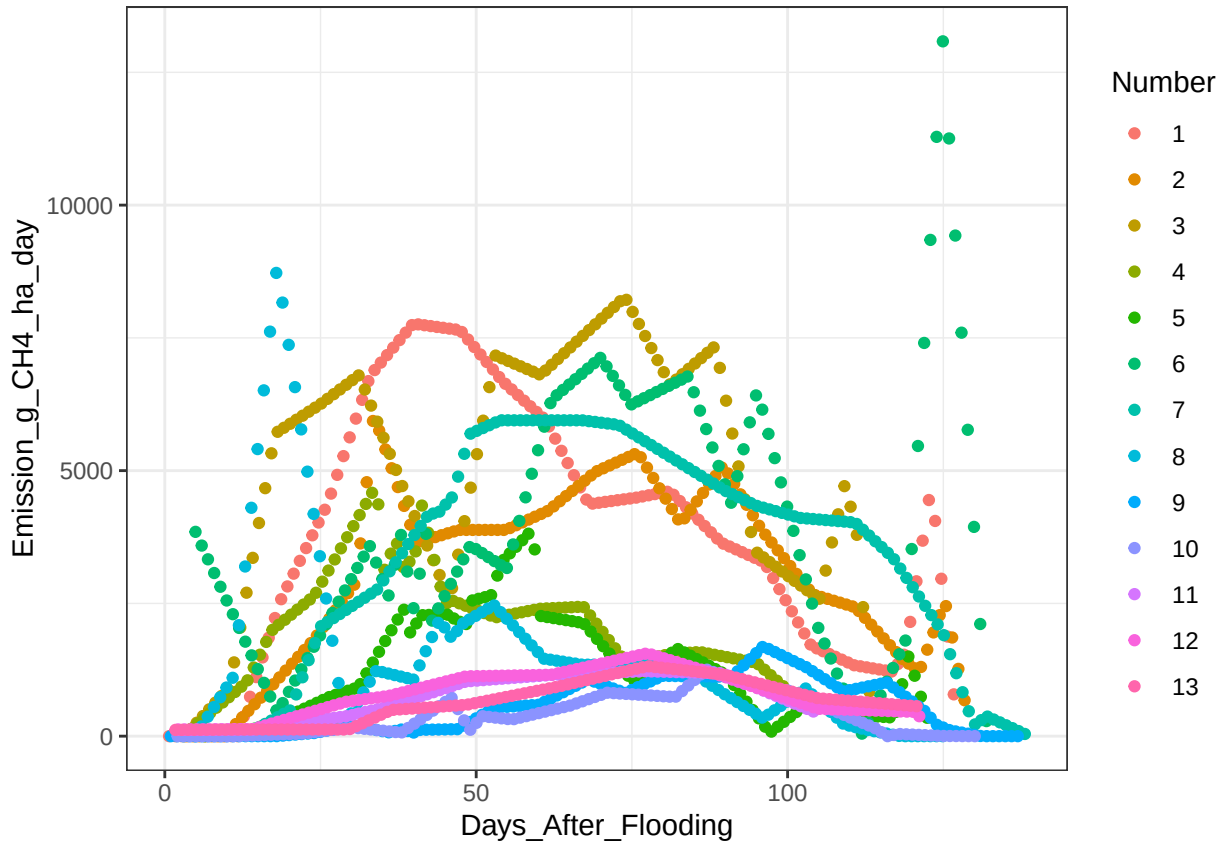
```
## Warning: <ggplot> %+% x was deprecated in ggplot2 4.0.0.
## i Please use <ggplot> + x instead.
## This warning is displayed once per session.
## Call ‘lifecycle::last_lifecycle_warnings()‘ to see where this warning was
## generated.
```



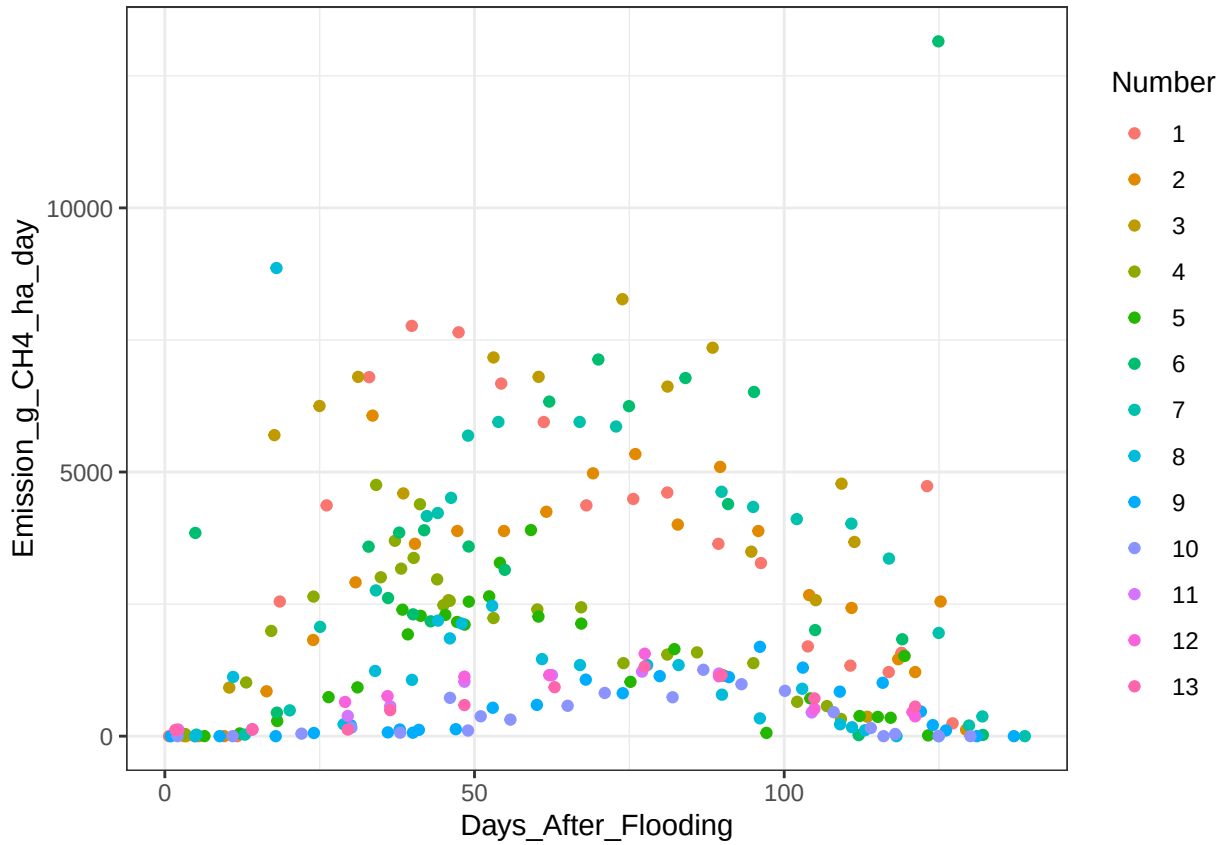

```
ggplot(master%>%filter(Number=="1"), aes(y=Emission_g_CH4_ha_day, x=Days_After_Flooding, color = Number)
```




```
ggplot(flux_interpolated, aes(y=Emission_g_CH4_ha_day, x=Days_After_Flooding, color = Number)) + geom_p
```




```
ggplot(master, aes(y=Emission_g_CH4_ha_day, x=Days_After_Flooding, color = Number)) + geom_point()
```



Correlation function (look at corr of Emission_g_CH4_ha_day with cumulative)

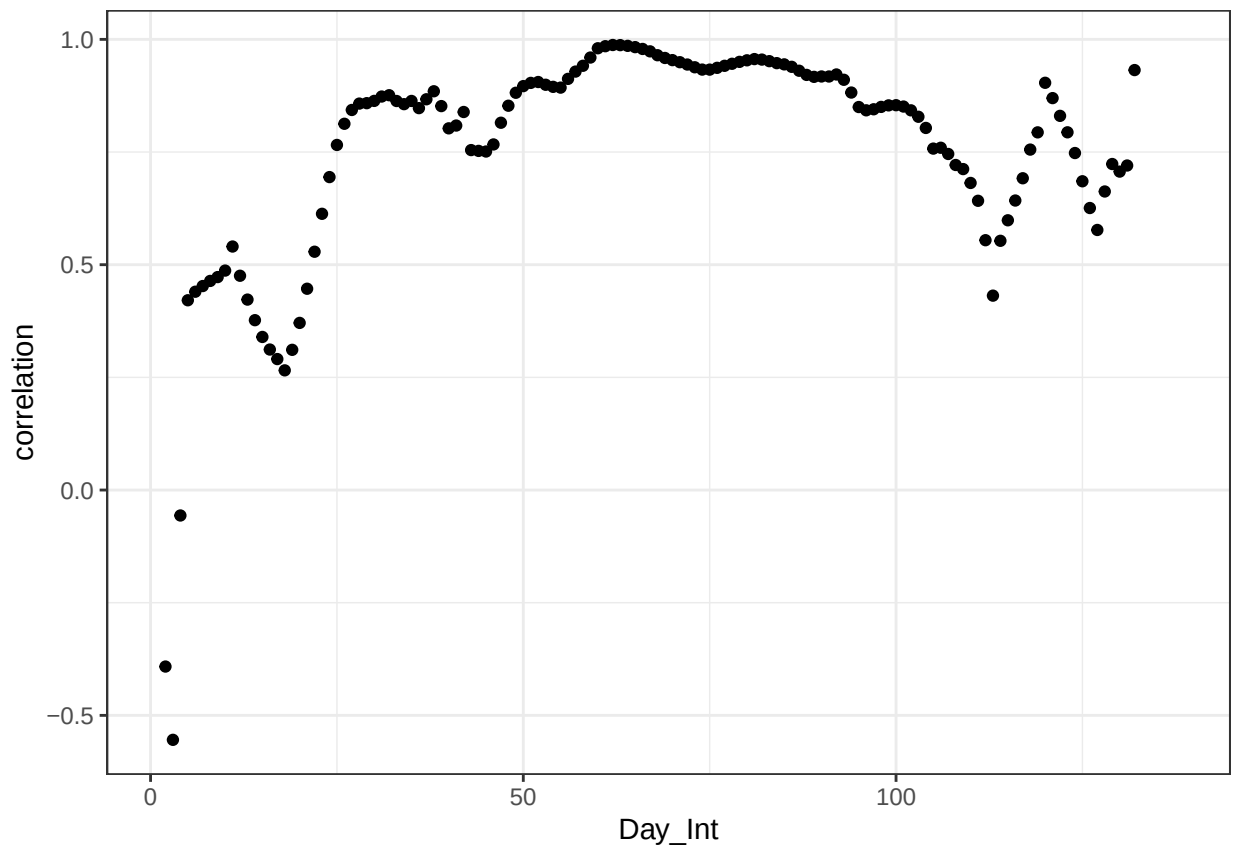
```
cor_emission_cumulative_by_day <- function(df,
                                           emission_col = "Emission_g_CH4_ha_day",
                                           cumulative_col = "Cumulative_seaonal_kg_CH4_ha",
                                           day_col = "Days_After_Flooding") {

  df %>%
    mutate(
      Day_Int = as.integer(round(.data[[day_col]])),
      Emission_num = as.numeric(.data[[emission_col]]),
      Cumulative_num = as.numeric(.data[[cumulative_col]])
    ) %>%
    group_by(Day_Int) %>%
    summarise(
      n = sum(!is.na(Emission_num) & !is.na(Cumulative_num)),

      correlation = if (n >= 3) {
        cor(Emission_num,
            Cumulative_num,
            use = "complete.obs",
            method = "pearson")
```

```
      } else NA_real_,

      p_value = if (n >= 3) {
        cor.test(Emission_num,
                 Cumulative_num,
                 method = "pearson")$p.value
      } else NA_real_,

      .groups = "drop"
    )
}
```

Try function

```
cor_results <- cor_emission_cumulative_by_day(flux_interpolated)

ggplot(cor_results, aes(y=correlation, x=Day_Int)) + geom_point()
```

```
## Warning: Removed 7 rows containing missing values or values outside the scale range
## (`geom_point()`).
```



Try R2 and regression

Function

```
find_best_predictive_day <- function(df,
                                     emission_col = "Emission_g_CH4_ha_day",
                                     cumulative_col = "Cumulative_seaonal_kg_CH4_ha",
                                     day_col = "Days_After_Flooding") {

  results <- df %>%
    mutate(
      Day_Int = as.integer(round(.data[[day_col]])),
      Emission_num = as.numeric(.data[[emission_col]]),
      Cumulative_num = as.numeric(.data[[cumulative_col]])
    ) %>%
    group_by(Day_Int) %>%
    summarise(
      n = sum(!is.na(Emission_num) & !is.na(Cumulative_num)),

      model = if (n >= 3) list(lm(Cumulative_num ~ Emission_num)) else list(NULL),

      R2 = if (!is.null(model[[1]])) summary(model[[1]])$r.squared else NA_real_,

      slope = if (!is.null(model[[1]])) coef(model[[1]])[2] else NA_real_,

      p_value = if (!is.null(model[[1]]))
        summary(model[[1]])$coefficients[2,4] else NA_real_,

      .groups = "drop"
    ) %>%
    select(-model)

  best_day <- results %>%
    filter(R2 == max(R2, na.rm = TRUE))

  list(
    all_days = results,
    best_day = best_day
  )
}
```

Run dataset

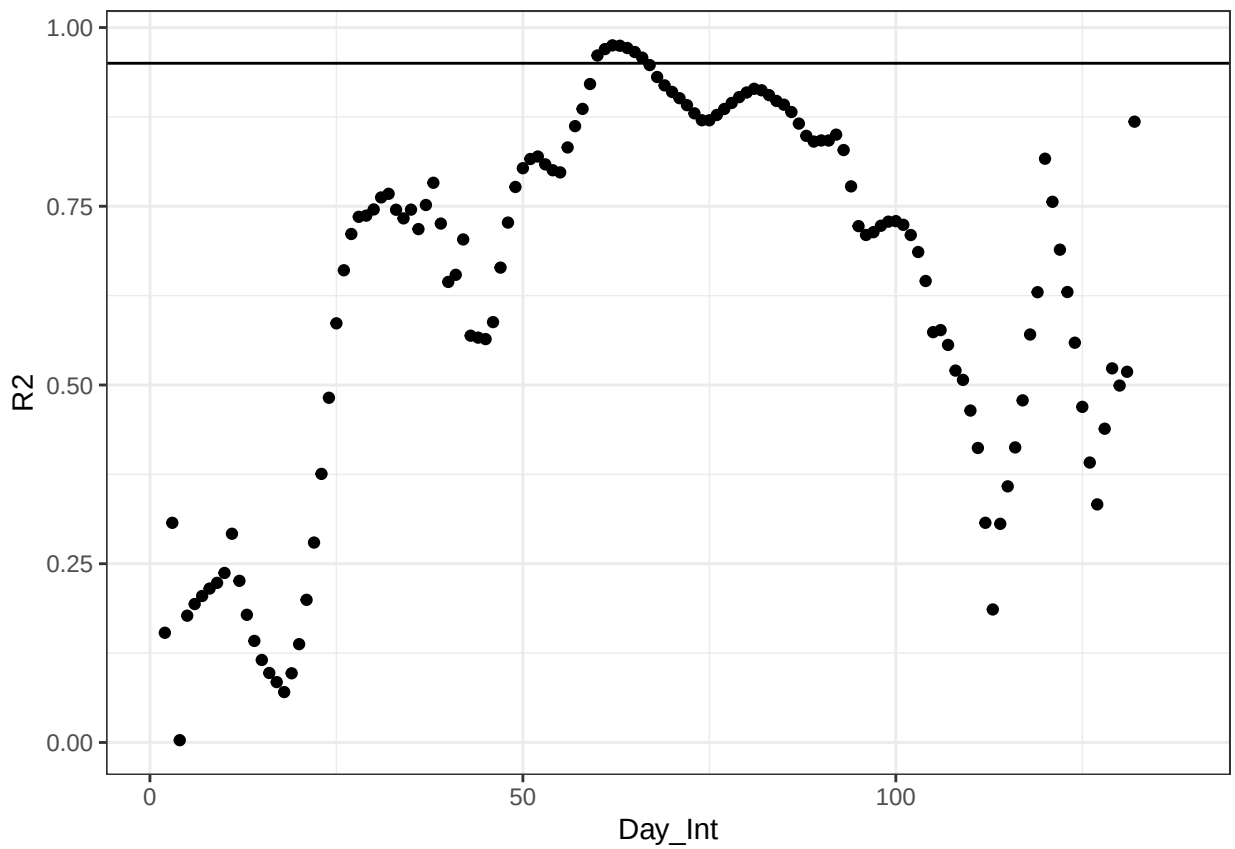
```
output <- find_best_predictive_day(flux_interpolated)

all_days <- output$all_days      # full summary
output$best_day      # day with highest R²
```

```
## # A tibble: 1 x 5
##   Day_Int     n    R2  slope  p_value
```

```
##     <int> <int> <dbl>  <dbl>    <dbl>
## 1      62    13 0.975 0.0785 3.63e-10
```

```
ggplot(all_days, aes(y=R2, x=Day_Int)) + geom_point() + geom_hline(yintercept=0.95) #+ geom_hline(yinte
```

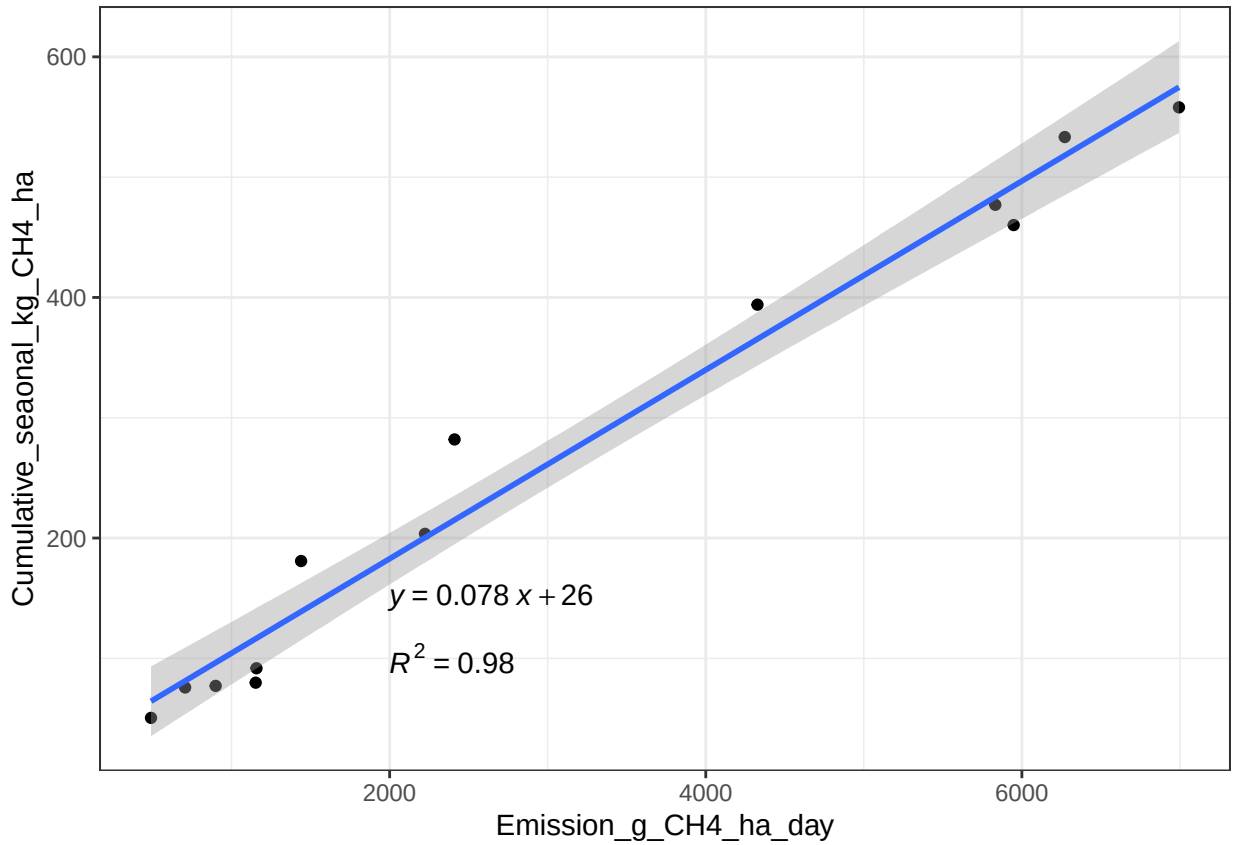
```
## Warning: Removed 7 rows containing missing values or values outside the scale range
## (`geom_point()`).
```



Use Day 62 to check

```
ggplot(data=flux_interpolated%>%filter(Day_Int==62), aes(x=Emission_g_CH4_ha_day, y=Cumulative_seaonal_
  geom_point()+
  geom_smooth(method = lm)+
  stat_regline_equation(aes(x=Emission_g_CH4_ha_day, y=Cumulative_seaonal_kg_CH4_ha,
    label =  paste(..rr.label..)),
    show.legend = FALSE,
    label.x = 2000,
    label.y = 100)+
  stat_regline_equation(aes(x=Emission_g_CH4_ha_day, y=Cumulative_seaonal_kg_CH4_ha,
    label = paste(..eq.label..)),
    show.legend = FALSE,
    label.x = 2000,
    label.y = 150)
```

```
## `geom_smooth()` using formula = 'y ~ x'
```



Let's use day 62 to predict seasonal cumulative emissions.

What if we just drew a triangle from day 62?

Essentially, if we just took the flux from day 62, linear interpolated between first and last days using the known growth period, and then summed daily emissions, how well will this predict seasonaly cumulative emissions?

Assume 130 days here for fun as a thought experiment

```
interpolate_day62_triangle <- function(df,
                                       day_col = "Days_After_Flooding",
                                       emission_col = "Emission_g_CH4_ha_day",
                                       group_col = "Number") {

  df %>%
    mutate(
      Day_Int = as.integer(round(.data[[day_col]])),
      Emission_num = as.numeric(.data[[emission_col]])
    ) %>%
    group_by(.data[[group_col]]) %>%
    summarise({

      # Extract ONLY day 62 flux
      y62 <- Emission_num[Day_Int == 62][1]
```

```
      if (is.na(y62)) {
        tibble(total_emission_0_130 = NA_real_)
      } else {

        days <- 0:130

        interp_values <- ifelse(
          days <= 62,
          y62 * (days / 62),
          y62 * (130 - days) / (130 - 62)
        )

        tibble(
          total_emission_0_130 = sum(interp_values)
        )
      }

    }, .groups = "drop")
}
```

```
day62_predict <- interpolate_day62_triangle(flux_interpolated) %>% mutate(total_emission_0_130_kg_ha = t
```

Join data

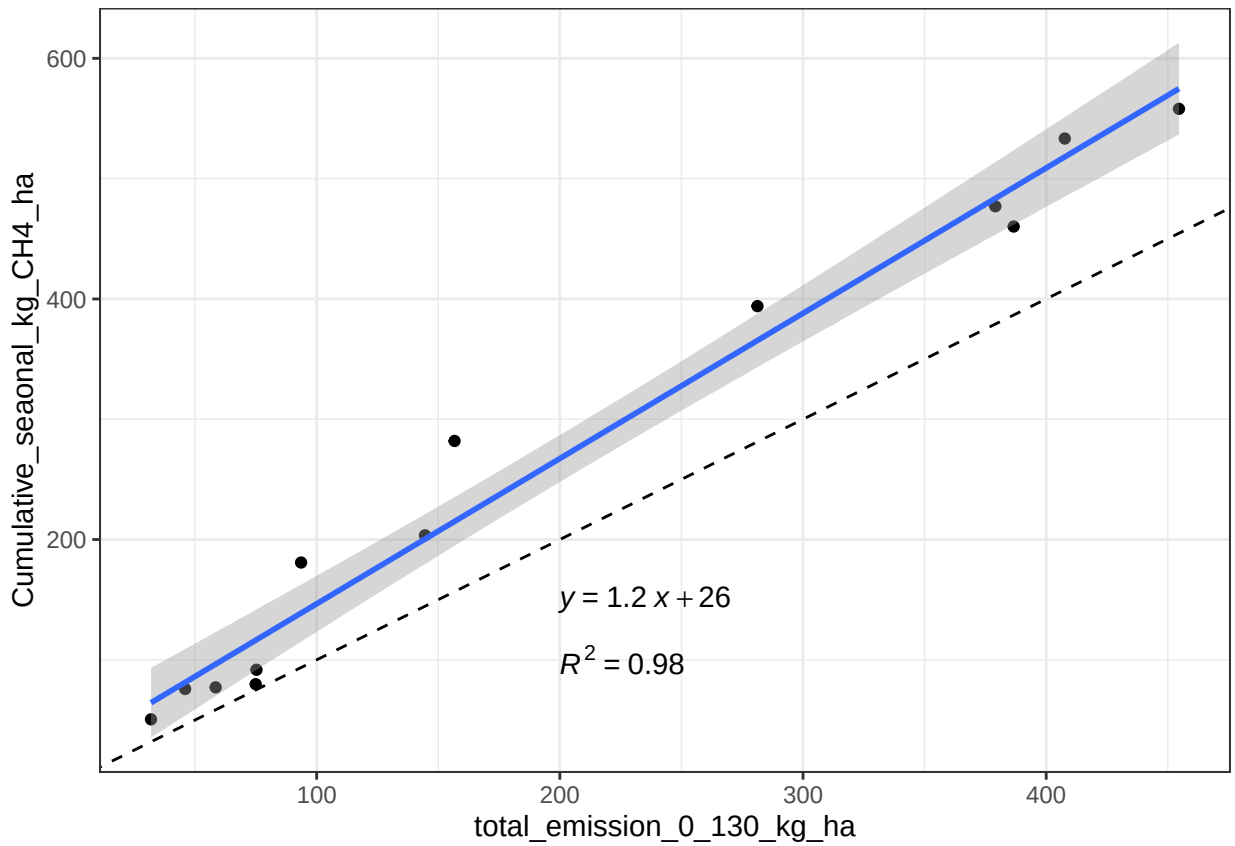
```
master_summary <- master %>%
  group_by(Number) %>%
  summarise(Cumulative_seaonal_kg_CH4_ha = mean(Cumulative_seaonal_kg_CH4_ha, na.rm = TRUE),
            .groups = "drop")

master_joined <-day62_predict %>%
  left_join(master_summary, by = "Number")

master_joined$diff_triangle <- master_joined$total_emission_0_130_kg_ha-master_joined$Cumulative_seaonal
master_joined$diff_triangle_percent <- 100*abs(master_joined$diff_triangle/master_joined$Cumulative_sea

mean(master_joined$diff_triangle_percent)
```

```
## [1] 27.2007
```

```
# generally, we see that this approach works, but it tends to underpredict emissions. But honestly, its

ggplot(master_joined, aes(y=Cumulative_seaonal_kg_CH4_ha, x=total_emission_0_130_kg_ha)) + geom_point()+
geom_smooth(method = lm)+
  stat_regline_equation(aes(x=total_emission_0_130_kg_ha, y=Cumulative_seaonal_kg_CH4_ha,
    label =  paste(..rr.label..)),
    show.legend = FALSE,
    label.x = 200,
    label.y = 100)+
  stat_regline_equation(aes(x=total_emission_0_130_kg_ha, y=Cumulative_seaonal_kg_CH4_ha,
    label = paste(..eq.label..)),
```

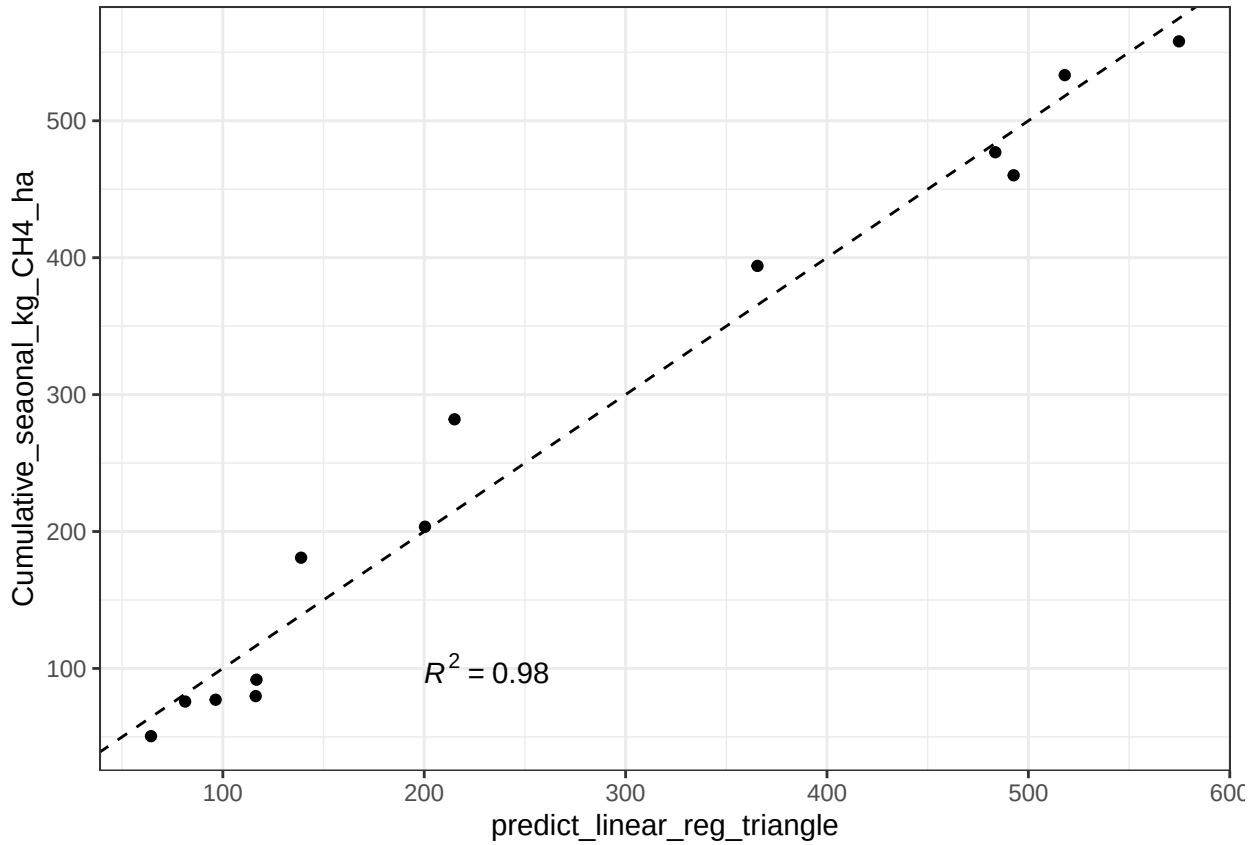
```
    show.legend = FALSE,
    label.x = 200,
    label.y = 150)+
    geom_abline(intercept = 0, slope = 1, linetype = "dashed")
```

```
## `geom_smooth()` using formula = 'y ~ x'
```



If we adjust the above output with linear regression, we get an average error in prediction by about 15%

```
lm_day62_triangle <- lm(Cumulative_seaonal_kg_CH4_ha~total_emission_0_130_kg_ha,master_joined)
master_joined$predict_linear_reg_triangle <- predict(lm_day62_triangle, master_joined)

ggplot(master_joined, aes(y=Cumulative_seaonal_kg_CH4_ha, x=predict_linear_reg_triangle)) + geom_point(
#geom_smooth(method = lm)+
  stat_regline_equation(aes(x=predict_linear_reg_triangle, y=Cumulative_seaonal_kg_CH4_ha,
    label =  paste(..rr.label..)),
    show.legend = FALSE,
    label.x = 200,
    label.y = 100)+
    geom_abline(intercept = 0, slope = 1, linetype = "dashed")
```




```
master_joined$diff_linear_reg_triangle <- master_joined$predict_linear_reg_triangle-master_joined$Cumula

master_joined$diff_linear_reg_triangle_diff <- 100*abs(master_joined$diff_linear_reg_triangle/master_joi


mean(master_joined$diff_linear_reg_triangle_diff)
```

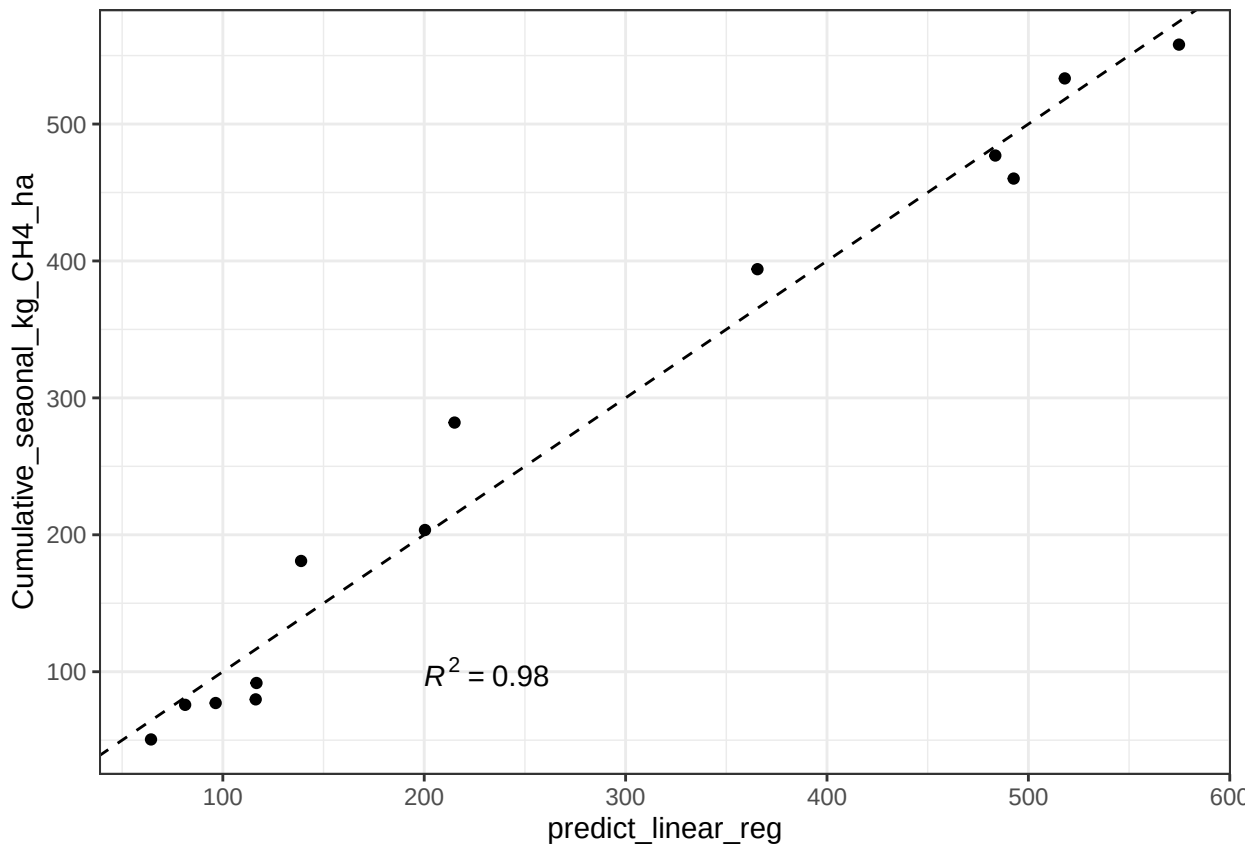
```
## [1] 15.57524
```

What is we used the linear regrssion of day 62 emission against seasonal cumulative to get a cumulative value? Actually it is the same answer as immediately above.

```
lm_day62 <- lm(Cumulative_seaonal_kg_CH4_ha~Emission_g_CH4_ha_day, flux_interpolated%>%filter(Day_Int==6


master_joined$predict_linear_reg <- predict(lm_day62, flux_interpolated%>%filter(Day_Int==62))
```

```
ggplot(master_joined, aes(y=Cumulative_seaonal_kg_CH4_ha, x=predict_linear_reg)) + geom_point()+
#geom_smooth(method = lm)+
  stat_regline_equation(aes(x=predict_linear_reg, y=Cumulative_seaonal_kg_CH4_ha,
    label =  paste(..rr.label..)),
```

```
    show.legend = FALSE,
    label.x = 200,
    label.y = 100)+
# stat_regline_equation(aes(x=predict_linear_reg, y=Cumulative_seaonal_kg_CH4_ha,
#    label = paste(..eq.label..)),
#    show.legend = FALSE,
#    label.x = 200,
#    label.y = 150)+
    geom_abline(intercept = 0, slope = 1, linetype = "dashed")
```




```
master_joined$diff_linear_reg <- master_joined$predict_linear_reg-master_joined$Cumulative_seaonal_kg_CH
master_joined$diff_linear_reg_percent_diff <- 100*abs(master_joined$diff_linear_reg/master_joined$Cumula


mean(master_joined$diff_linear_reg_percent_diff)
```

```
## [1] 15.57524
```